DevLife Manager

개발자 생활 관리 시스템

기술 설계서

Tech Stack

Category	Technology Stack
Backend	Java, Spring Boot, Spring Security, JPA (Hibernate)
Frontend	Vue.js 3, JavaScript, HTML5, CSS3, Chart.js
Database	PostgreSQL 15
Infra/DevOps	Git, GitHub

프로젝트 기간	202604~202608
작성자	조수진 8년 차 풀스택 개발자
GitHub	https://github.com/SUJINCHO/devlife-manager
Email	publiccho98@gmail.com

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	개정 이력	조수진	26-04-10	조수진	

버전	개정 일자	대상 시트	개정 내용 요약	개정자
v1.0	26-04-10	전체	초기 작성	조수진

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	요구사항	조수진	26-04-10	조수진	

1. 개요

1.1. 프로젝트 명

DevLife Manager (개발자 생활 관리 시스템)

1.2. 목적

개발자의 일상(학습, 작업)을 통합 관리하고,
데이터 기반으로 생산성을 분석하여 자기 개선을 돕는 웹 애플리케이션을 구축한다.

1.3. 배경

- 개발자는 학습 + 작업 + 생활 관리를 동시에 필요로 함
- 단순 기록이 아닌 데이터 기반 분석 및 피드백 시스템 필요

1.4. 기대효과

- 개인 생산성 향상
- 목표 달성률 증가

2. 기능 요구사항

ID	카테고리	기능명	기능 개요	우선순위
FR-AUTH-001	사용자 관리	회원 가입	사용자가 계정을 생성한다	상
FR-AUTH-002	사용자 관리	로그인	사용자가 로그인한다	상
FR-AUTH-003	사용자 관리	회원 정보 변경	사용자정보를 변경한다	중
FR-TASK-001	Task 관리	Task 생성	할 일을 등록한다	상
FR-TASK-002	Task 관리	Task 상태 변경	Task 상태를 변경한다	상
FR-TASK-003	Task 관리	Task 조회	할 일을 등록한다	상
FR-ACT-001	활동 기록	활동 기록 생성	활동을 기록한다	상
FR-ACT-003	활동 기록	활동 조회	날짜별 활동 조회	중
FR-DASH-001	대시보드	요약정보 조회	주요 데이터 요약 표시	상
FR-STAT-001	통계	활동 통계	활동 시간 그래프 제공	중
FR-SCORE-001	생산성	점수 계산	생산성 점수를 계산한다	상

3. 비기능 요구사항

ID	카테고리	항목	내용 및 목표 기준	우선순위
NFR-PERF-001	성능	API 응답 속도	API 응답 시간 500ms 이하	상
NFR-PERF-002	성능	대시보드 로딩	초기 로딩 시간 2초 이하	상
NFR-SEC-001	보안	인증	JWT 인증 적용 모든 API 인증 필요	상
NFR-SEC-002	보안	비밀번호 암호화	사용자 비밀번호 보호 BCrypt 적용	상
NFR-DATA-001	데이터	무결성	데이터 정합성 유지 트랜잭션 처리	상
NFR-LOG-001	로그	로그 관리	시스템 로그 기록 에러 및 요청 로그 저장	중
NFR-MTN-001	유지보수	코드 구조	계층형 아키텍처 Controller/Service/Repo 분리	상
NFR-UX-001	사용성	UI/UX	사용자 편의성 직관적 UI	중

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	요구사항 상세	조수진	26-04-10	조수진	

1. FR-AUTH-001 회원가입

항목	내용
기능명	회원가입
개요	사용자가 계정을 생성한다
입력값	email, password, name
처리 로직	1. 이메일 중복 확인
	2. 비밀번호 BCrypt 암호화
	3. 사용자 정보 DB 저장
출력값	회원가입 성공 여부
예외 처리	이메일 중복 시 오류 반환
비고	이메일 형식 검증 필요

2. FR-AUTH-002 로그인

항목	내용
기능명	로그인
개요	사용자를 인증하고 토큰을 발급한다
입력값	email, password
처리 로직	1. 사용자 조회
	2. 비밀번호 검증
	3. JWT 토큰 생성
출력값	accessToken, tokenType (Bearer), expiresIn
예외 처리	이메일 또는 비밀번호 불일치 시 오류 반환
	존재하지 않는 사용자일 경우 오류 반환
비고	토큰 만료 시간 설정 필요

3. FR-AUTH-003 회원 정보 변경

항목	내용
기능명	회원 정보 변경
개요	등록 되어있는 사용자의 정보를 변경한다
입력값	email, password, name
처리 로직	1. 회원정보 조회
	2. 이메일 중복 확인
	3. 비밀번호 BCrypt 암호화
	4. DB 업데이트
출력값	회원 정보 변경 성공 여부
예외 처리	이메일 중복 시 오류 반환
비고	이메일 형식 검증 필요

4. FR-TASK-001 Task 생성

항목	내용
기능명	Task 생성
개요	사용자가 할 일을 등록한다
입력값	title, status, priority, dueDate ※ userId는 JWT 토큰에서 추출
처리 로직	1. 입력값 검증
	2. Task 정보 DB 저장
출력값	생성된 Task 정보
예외 처리	필수값 누락 시 오류 반환
비고	priority 기본값 설정 가능

5. FR-TASK-002 Task 상태 변경

항목	내용
기능명	Task 상태 변경
개요	사용자가 Task 상태를 변경한다
입력값	taskId, status
처리 로직	1. Task 조회
	2. 상태 변경
	3. DB 업데이트
출력값	변경된 Task 정보
예외 처리	Task 미존재 시 오류 반환
비고	상태값: TODO / DOING / DONE

6. FR-TASK-003 Task 조회

항목	내용
기능명	Task 조회
개요	사용자의 할일을 기간별로 조회한다
입력값	targetMonth ※ userId는 JWT 토큰에서 추출
처리 로직	1. 기간 조건으로 tasks 조회
	2. 결과 리스트 반환
출력값	할일 리스트
예외 처리	날짜 형식 오류 시 오류 반환
비고	페이징 처리 고려(Query Params: page, size)

7. FR-ACT-001 활동 기록 생성

항목	내용
기능명	활동 기록 생성
개요	사용자가 활동을 기록한다
입력값	type, duration, memo, date ※ userId는 JWT 토큰에서 추출
처리 로직	1. 입력값 검증
	2. Activity 정보 DB 저장
출력값	생성된 활동 기록
예외 처리	duration 0 이하 입력 시 오류 반환
비고	type: STUDY / CODING / ETC

8. FR-ACT-002 활동 조회

항목	내용
기능명	활동 조회
개요	사용자의 활동을 기간별로 조회한다
입력값	targetMonth ※ userId는 JWT 토큰에서 추출
처리 로직	1. 기간 조건으로 Activity 조회
	2. 결과 리스트 반환
출력값	활동 리스트
예외 처리	날짜 형식 오류 시 오류 반환
비고	페이징 처리 고려(Query Params: page, size)

9. FR-DASH-001 대시보드 조회

항목	내용
기능명	요약 정보 조회
개요	주요 데이터를 요약하여 조회한다
입력값	※ userId는 JWT 토큰에서 추출
처리 로직	1. 오늘 Task 조회
	2. 활동 시간 합계 계산
	3. 생산성 점수 조회
출력값	대시보드 데이터 객체
예외 처리	없음
비고	캐싱 적용 고려 가능

10. FR-STAT-001 활동 통계

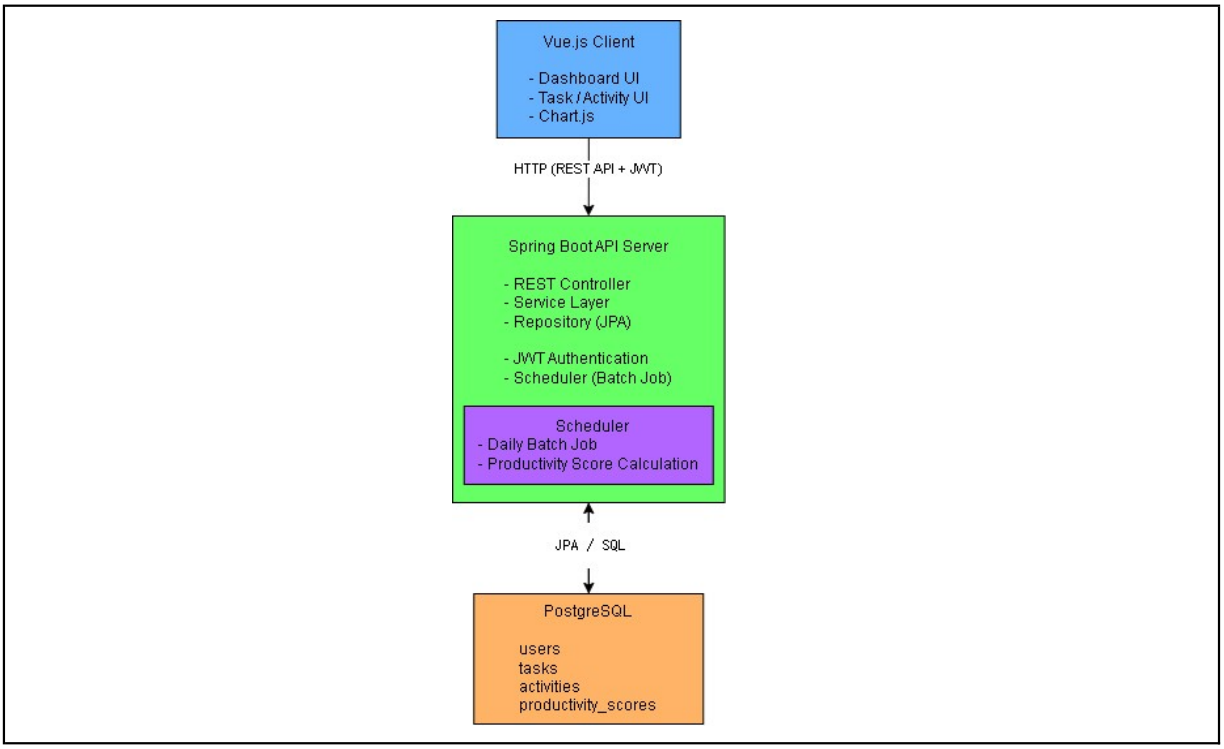
항목	내용
기능명	활동 통계
개요	활동 시간 데이터를 통계로 제공한다
입력값	※ userId는 JWT 토큰에서 추출
처리 로직	1. 기간별 Activity 집계
	2. 날짜별 데이터 생성
출력값	날짜별 활동 시간 리스트
예외 처리	없음
비고	Chart.js 연동 예정

11. FR-SCORE-001 생산성 점수 계산

항목	내용
기능명	생산성 점수 계산
개요	사용자 생산성 점수를 계산한다
입력값	※ userId는 JWT 토큰에서 추출
처리 로직	1. 활동 시간 조회
	2. 완료 Task 수 조회
	3. 점수 계산 로직 적용(활동시간 * 가중치 + 완료 Task 수 * 가중치)
출력값	생산성 점수
예외 처리	데이터 없을 경우 기본값 처리
비고	생산성 점수는 배치 작업을 통해 주기적으로 계산된다.

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	시스템 아키텍처	조수진	26-04-10	조수진	

1. 아키텍처 다이어그램



2. 시스템 구성 개요

본 시스템은 클라이언트-서버 구조 기반의 웹 애플리케이션으로,
Vue.js 기반의 프론트엔드와 Spring Boot 기반의 백엔드 API 서버,
PostgreSQL 데이터베이스로 구성된다.

클라이언트(Vue.js)는 사용자 인터페이스를 제공하며,
REST API를 통해 백엔드 서버와 통신한다.

백엔드(Spring Boot)는 REST Controller, Service Layer,
Repository(JPA)로 구성된 계층형 아키텍처를 기반으로
비즈니스 로직을 처리한다.

데이터는 PostgreSQL에 저장되며,
JPA를 통해 객체-관계 매핑 방식으로 관리된다.

3. 아키텍처 설계 의도

JWT 기반 인증을 적용한 이유는 서버 상태를 최소화하여 Stateless 구조를 유지하고,
세션 기반 인증에서 발생하는 서버 간 세션 공유 비용을 제거하기 위함이다.
이를 통해 로드밸런싱 환경에서도 별도의 세션 동기화 없이
수평 확장이 가능하도록 설계하였다.

또한 계층형 아키텍처(Controller-Service-Repository)를 적용하여
비즈니스 로직과 데이터 접근을 분리함으로써 유지보수성과 확장성을 확보하였다.

4. 성능 설계 전략

본 시스템은 단순 CRUD 기반 구조가 아닌,
조회 성능 최적화를 위해 사전 계산(Pre-computation) 구조를 채택하였다.

생산성 점수는 실시간 처리 대신 Scheduler 기반 배치를 통해
일 단위로 계산되며, 결과는 별도 테이블(productivity_scores)에 저장된다.

5. Trade-off & 운영 고려

배치 처리 방식은 데이터가 실시간이 아닌 지연될 수 있는 단점이 존재한다.
하지만 본 시스템은 개인 생산성 관리 서비스로,
실시간성이 절대적으로 요구되지 않기 때문에
일 단위 지연은 허용 가능한 수준으로 판단하였다.

또한 배치 실행 로그를 관리하여 실패 시 재처리가 가능하도록 설계하였다.

6. 핵심 데이터 흐름 (Data Flow)

1. 사용자는 Vue.js 클라이언트를 통해 로그인 및 주요 기능(Task, Activity 등)을 수행한다.

2. 클라이언트는 JWT 토큰을 포함한 HTTP 요청을 Spring Boot API 서버로 전송한다.

3. 백엔드 서버는 JWT 인증을 통해 사용자를 검증한 후,
   Controller → Service → Repository 계층을 통해 비즈니스 로직을 처리한다.

4. 데이터는 JPA를 통해 PostgreSQL 데이터베이스에 저장되거나 조회된다.

5. 조회된 데이터는 다시 API 응답으로 클라이언트에 전달되며,
   클라이언트는 이를 기반으로 화면을 렌더링한다.

6. Scheduler는 일정 주기(일 단위)로 실행되며,
   Activity, Task 데이터를 조회하여
   생산성 점수를 계산한 후 별도 테이블에 저장한다.

7. 대시보드 조회 시, 클라이언트는 집계된 생산성 점수 및
   활동 데이터를 API를 통해 조회하여 사용자에게 시각적으로 제공한다.

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	기술 의사결정서	조수진	26-04-10	조수진	

1. 개요
본 프로젝트는 리스크를 최소화하기 위해 MVP 기반으로 개발 범위를 제한하고,
핵심 기능을 우선 구현하는 전략을 적용하였다.

2. 기술 의사결정 상세 (Decision Log)

항목	선택 기술	선정 이유
Backend	Spring Boot	Spring Boot를 선택한 이유는 REST API 구현이 용이하며, Spring Security(JWT), Scheduler 등과의 통합이 자연스럽고 하나의 프레임워크 내에서 일관된 방식으로 아키텍처를 구성할 수 있기 때문이다. 특히 인증, 배치, 데이터 접근 계층을 동일한 생태계 내에서 구성할 수 있어 구조적 일관성을 유지하기 용이하다. Trade-off: 경량 프레임워크 대비 초기 설정 비용과 러닝 커브가 존재하지만, 장기적인 유지보수성과 확장성을 고려하여 선택하였다.
Frontend	Vue.js	Vue.js를 선택한 이유는 비교적 학습 곡선이 완만하고, 컴포넌트 기반 구조를 통해 UI를 효율적으로 구성할 수 있기 때문이다. 또한 상태 관리 및 화면 단위 분리가 용이하여 대시보드, Task, Activity 등의 기능을 모듈화하기에 적합하다. Trade-off: 대규모 애플리케이션에서는 구조 관리가 어려워질 수 있으나, 본 프로젝트는 MVP 범위로 제한하여 복잡도를 제어하였다.
Database	PostgreSQL	PostgreSQL을 선택한 이유는 안정성과 확장성이 뛰어나며, ACID를 준수하는 트랜잭션 처리로 데이터 무결성을 보장할 수 있기 때문이다. 또한 다양한 인덱스 및 통계 기능을 제공하여 대시보드 및 통계 조회 성능 최적화에 유리하다. Trade-off: 초기 설정 및 튜닝이 필요할 수 있으나, 오픈소스 기반으로 비용 효율성과 기능성을 고려하여 선택하였다.

Authentication	JWT	JWT 기반 인증을 적용한 이유는 서버 상태를 최소화하여 Stateless 구조를 유지하고, 세션 기반 인증에서 발생하는 서버 간 세션 공유 비용을 제거하기 위함이다. 이를 통해 로드밸런싱 환경에서도 별도의 세션 동기화 없이 수평 확장이 가능하도록 설계하였다. Trade-off: 토큰 기반 인증은 토큰 탈취 시 보안 위험이 존재하므로, 토큰 만료 시간 설정 및 인증 필터를 통해 이를 보완하였다.
ORM	JPA	JPA를 선택한 이유는 객체 중심의 개발이 가능하여 비즈니스 로직과 데이터 모델 간의 매핑을 자연스럽게 처리할 수 있기 때문이다. 또한 반복적인 CRUD SQL 작성을 줄여 개발 생산성을 향상시키고, 엔티티 기반 설계를 통해 유지보수성을 높일 수 있다. Trade-off: 복잡한 조회 쿼리의 경우 성능 저하가 발생할 수 있으므로, 필요 시 JPQL 또는 Native Query를 병행하여 사용하도록 고려하였다.
Batch	Spring Scheduler	Spring Scheduler를 선택한 이유는 Spring 환경과 자연스럽게 통합되며, 별도의 배치 프레임워크 없이도 간단한 주기 작업을 구현할 수 있기 때문이다. 생산성 점수 계산과 같은 주기적 작업에 적합하며, 구현 복잡도를 낮추는 데 유리하다. Trade-off: 대규모 배치 처리에는 한계가 있으므로, 향후 확장 시 Spring Batch 도입을 고려할 수 있다.

3. 리스크 관리

대상	잠재적 리스크	대응 방안
일정 지연	개인 프로젝트로 개발 시간이 제한적임	MVP 범위 우선 개발, 우선순위 기반 기능 구현
기능 과다	과도한 기능 추가로 개발 복잡도 증가	핵심 기능(Task, Activity, Score) 중심 개발
인증 보안	JWT 토큰 관리 미흡 시 보안 취약점 발생	토큰 만료 시간 설정 및 인증 필터 적용
데이터 정합성	잘못된 데이터 입력으로 통계 오류 발생	입력값 검증 및 트랜잭션 처리 적용
성능 문제	통계 및 대시보드 조회 시 성능 저하 가능	인덱스 설계 및 생산성 점수 별도 테이블 관리
배치 처리 오류	Scheduler 실행 실패 시 데이터 누락 가능	예외 처리 및 로그 기록, 재실행 가능 구조 고려
확장성 부족	초기 설계가 확장성을 고려하지 않을 경우 유지보수 어려움	계층형 구조 및 도메인 분리 설계 적용
기술 학습 부담	새로운 기술 사용 시 개발 속도 저하	검증된 기술 스택(Spring, Vue) 사용

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	데이터 모델링 (ERD)	조수진	26-04-10	조수진	

1. 엔티티 관계도 (ERD)

2. 엔티티 목록

물리명	논리명	상세
users	사용자	사용자 계정 및 인증 정보 관리
tasks	할 일	사용자의 할 일 정보 관리
activities	활동 기록	사용자의 학습/코딩 등 활동 데이터 기록
productivity_scores	생산성 점수	사용자 활동 기반 생산성 점수 저장

3. 데이터 모델링 설계 주안점

항목	설계 전략	기대 효과
식별자 설계	PK는 BIGINT AUTO_INCREMENT 사용	성능 향상 및 JOIN 처리 최적화
사용자 중심 구조	모든 테이블에 user_id FK 적용	사용자 단위 데이터 관리 용이
도메인 분리	Task, Activity, Score 테이블 분리	역할 분리 및 유지보수성 향상
ENUM 구조	status, type, priority는 문자열로 저장하며,애플리케이션 레벨에서 Enum으로 관리한다.	데이터 일관성 및 가독성 향상
시간 데이터 관리	created_at, updated_at 필드 공통 적용	변경 이력 추적 가능

통계 데이터 분리	생산성 점수 별도 테이블 관리	대시보드 조회 시 집계 쿼리 비용을 줄이고,읽기 성능을 최적화하기 위함
확장성 고려	Expense 기능 단순화 (MVP), 추후 확장 가능	개발 기간 단축 및 유연한 확장
인덱스 고려	주요 조회 패턴을 고려하여 복합 인덱스 설계 (user_id, date) (user_id, status)	조회 성능 향상 (통계/대시보드)

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	테이블 상세설계	조수진	26-04-10	조수진	

1. users

물리명	논리명	타입	Null	제약 조건	설명
user_id	유저 아이디	BIGINT	NO	PK, AUTO_INCREMEN	사용자 고유 식별자
email	이메일	VARCHAR(255)	NO	UNIQUE	로그인용 이메일
password	비밀번호	VARCHAR(255)	NO	-	BCrypt 암호화 저장
name	이름	VARCHAR(100)	YES	-	사용자 이름
created_at	등록일	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	생성일시
updated_at	갱신일	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	수정일시

2. tasks

컬럼명	논리명	타입	Null	제약 조건	설명
task_id	Task 아이디	BIGINT	NO	PK, AUTO_INCREMEN	Task 고유 식별자
user_id	유저 아이디	BIGINT	NO	FK	USER 테이블 참조
title	제목	VARCHAR(255)	NO	-	할 일 제목
status	상태	VARCHAR(20)	NO	-	TODO, DOING, DONE
priority	우선순위	VARCHAR(20)	NO	-	LOW, MEDIUM, HIGH
due_date	마감일	DATE	YES	-	Task 마감일
created_at	등록일	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	생성일시
updated_at	갱신일	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	수정일시

3. activities

컬럼명	논리명	타입	Null	제약 조건	설명
activity_id	활동 아이디	BIGINT	NO	PK, AUTO_INCREMEN	활동 고유 식별자
user_id	유저 아이디	BIGINT	NO	FK	USER 테이블 참조
type	활동 유형	VARCHAR(20)	NO	-	STUDY, CODING, ETC
duration	활동 시간	INT	NO	-	분 단위 시간
memo	메모	VARCHAR(500)	YES	-	활동 내용 설명
date	활동 날짜	DATE	NO	-	활동 수행 날짜
created_at	등록일	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	생성일시

4. productivity_scores

컬럼명	논리명	타입	Null	제약 조건	설명
prod_id	점수 아이디	BIGINT	NO	PK, AUTO_INCREMEN	점수 고유 식별자
user_id	유저 아이디	BIGINT	NO	FK	USER 테이블 참조
score	점수	INT	NO	-	생산성 점수
date	날짜	DATE	NO	-	점수 기준 날짜
created_at	등록일	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	생성일시

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	API 명세서	조수진	26-04-10	조수진	

1. FR-AUTH-001 회원가입

POST /api/auth/signup

1.1. Request

```
{
  "email": "test@example.com",
  "password": "1234",
  "name": "홍길동"}
```

1.2. Response

```
{
  "success": true,
  "message": "회원가입 성공"
}
```

1.3. Error

```
{
  "success": false,
  "code": "EMAIL_ALREADY_EXISTS",
  "message": "이미 존재하는 이메일입니다."
}
```

2. FR-AUTH-002 로그인

POST /api/auth/login

2.1. Request

```
{
  "email": "test@example.com",
  "password": "1234"
}
```

2.2. Response

```
{
  "accessToken": "jwt_token",
  "tokenType": "Bearer",
  "expiresIn": 3600
}
```

2.3. Error

```
{
  "code": "INVALID_CREDENTIALS",
  "message": "이메일 또는 비밀번호가 올바르지 않습니다."
}
```

3. FR-AUTH-003 회원 정보 변경
POST /api/auth/signup
1.1. Request

```
{
  "email": "test@example.com",
  "password": "1234",
  "name": "홍길동"}
```

1.2. Response

```
{
  "success": true,
  "message": "회원가입 성공"
}
```

1.3. Error

```
{
  "success": false,
  "code": "EMAIL_ALREADY_EXISTS",
  "message": "이미 존재하는 이메일입니다."
}
```

4. FR-TASK-001 Task 생성
POST /api/tasks
4.1. Request

```
{
  "title": "Spring 공부",
  "status": "TODO",
  "priority": "HIGH",
  "dueDate": "2026-04-20"
}
```

4.2. Response

```
{
  "id": 1,
  "title": "Spring 공부",
  "status": "TODO",
  "priority": "HIGH",
  "dueDate": "2026-04-20"
}
```

5. FR-TASK-002 Task 상태 변경
PATCH /api/tasks/{taskId}/status
5.1. Request

```
{
  "status": "DONE"
}
```

5.2. Response

```
{
  "id": 1,
  "status": "DONE"
}
```

6. FR-TASK-003 Task 조회

GET /api/tasks

6.1. Query Params

```
targetMonth=2026-04
```

6.2. Response

```
{
  "content": [
    {
      "id": 1,
      "title": "공부",
      "status": "TODO",
      "priority": "HIGH",
      "dueDate": "2026-04-20"
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 100
}
```

7. FR-ACT-001 활동 기록 생성

POST /api/activities

7.1. Request

```
{
  "type": "STUDY",
  "duration": 120,
  "memo": "Spring Boot 공부",
  "date": "2026-04-18"
}
```

7.2. Response

```
{
  "id": 1,
  "type": "STUDY",
  "duration": 120,
  "memo": "Spring Boot 공부",
  "date": "2026-04-18"
}
```

8. FR-ACT-002 활동 조회

GET /api/activities

8.1. Query Params

```
targetMonth=2026-04
```

8.2. Response

```
{
  "content": [
    {
      "date": "2026-04-18",
      "type": "STUDY",
      "duration": 120
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 100
}
```

9. FR-DASH-001 대시보드 조회

GET /api/dashboard

9.1. Response

```
{
  "tasks": [
    { "title": "공부", "status": "TODO" },
    { "title": "운동", "status": "DOING" }
  ],
  "studyTime": 120,
  "codingTime": 90,
  "productivityScore": 85
}
```

※ 대시보드 응답은 추후 확장을 고려하여 변경될 수 있다.

10. FR-STAT-001 활동 통계

GET /api/stats/activities

10.1. Query Params

```
targetMonth=2026-04
```

10.2. Response

```
[
  {
    "date": "2026-04-01",
    "totalDuration": 100
  }
]
```

11. FR-SCORE-001 생산성 점수 계산

GET /api/score

11.1. Response

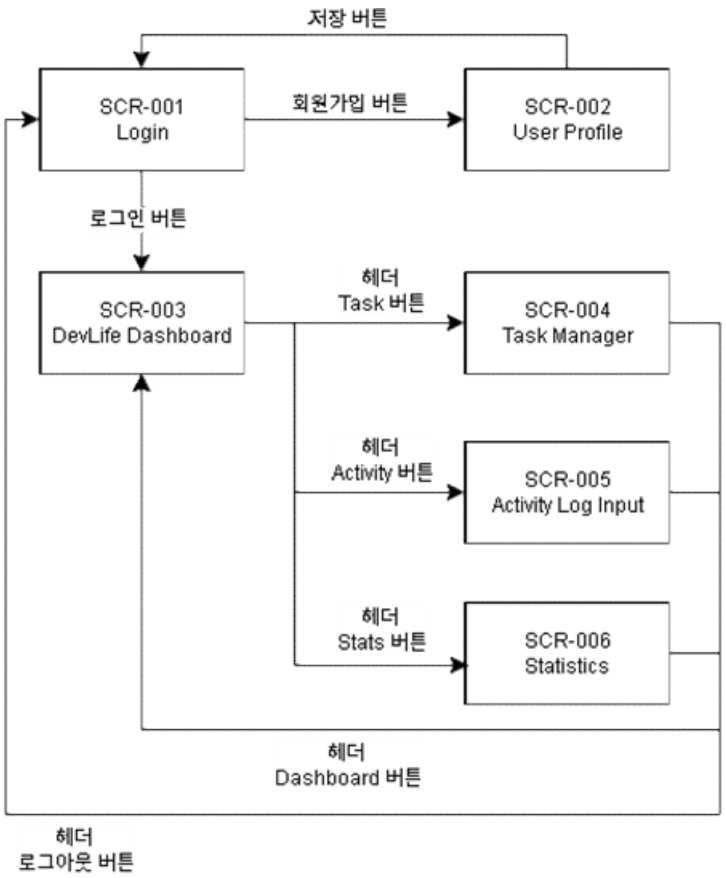
```
{
  "score": 85
}
```

DevLife Manager	Document	작성자	작성 날짜	수정자	수정 날짜
	화면 설계	조수진	26-04-10	조수진	

1. 화면일람

화면 ID	화면명	URL	설명	설명
SCR-001	Login	/login	사용자 로그인	사용자 로그인 정보 입력
SCR-002	User Profile	/profile	사용자 정보 조회	사용자 회원가입, 정보 조회, 수정 통괄
SCR-003	DevLife Dashboard	/dashboard	사용자 요약 정보	Task/Activity/Score 조회
SCR-004	Task Manager	/tasks	할 일 관리	생성, 상태 변경, 목록 조회
SCR-005	Activity Log Input	/activities	활동 기록	입력, 조회
SCR-006	Statistics	/statistics	통계	그래프 조회

2. 화면 천이도



3. 화면 레이아웃

3.1. 헤더

(LOGO) 시스템명 [Dashboard] (active) [Task] [Activity] [Stats] 홍길동 님 | 로그아웃

3.4. SCR-001 Login

Login

이메일: [test@example.com]
비밀번호 : [1234]

[로그인]
[회원가입]

3.5. SCR-002 User Profile

User Profile

이메일: [test@example.com]
비밀번호 : [1234]
이름: [홍길동]

[저장]

3.4. SCR-003 DevLife Dashboard

DevLife Dashboard

[오늘의 Task]
- 공부 (TODO)
- 운동 (DOING)

[오늘의 활동]
공부 시간: 120분
코딩 시간: 90분

[생산성 점수]
⭐ 85점

[그래프]
- 활동 시간 추이
(Chart.js)

3.5. SCR-004 Task Manager

Task Manager

제목: 독서
상태 : [TODO ▼]
무선순위: [HIGH ▼]
마감일: [26-03-10]

[저장]

[Task List] 기간: [26-04 ▼] [상태변경]

제목	상태	무선순위	마감일
공부	TODO▼	HIGH	26-03-10
운동	DOING▼	MEDIUM	26-03-10

3.6. SCR-005 Activity Log Input

Activity Log Input

유형: [STUDY ▼]
시간: [120] 분
날짜: [2026-04-18]
메모: [Spring Boot 공부]

[저장]

[최근기록] 기간: [26-04 ▼]

활동 날짜	활동 유형	활동 시간	메모
26-03-1(	STUDY	120분	spring공부
26-03-09	CODING	90분	A프로젝트

3.7. SCR-006 Statistics

Statistics

기간: [26-04 ▼]

[그래프]
- 일별 공부 시간
(Chart.js)
- 생산성 점수 추이
(Chart.js)